UNIVERSIDAD PRIVADA DE TACNA

ormalsize**FACULTAD DE INGENIERÍA**

ormalsize**ESCUELA DE INGENIERÍA DE SISTEMAS**



Guía Práctica de Laboratorio

“Implementación de Room Database en Kotlin”

Que se presenta para el curso:

“SOLUCIONES MOVILES I”

Docente:

Mag. Alberto Johnatan Flor Rodríguez

TACNA – PERÚ

2026

Índice General

Introducción 3
Guía de Laboratorio Nº 06 “Implementación de Room Database en Kotlin” 4
1. Información sobre el evento práctico 4
1.1. Título del evento práctico 4
1.2. Objetivos 4
1.3. Tiempo de duración (horas) 4
1.4. Resultados de Aprendizaje (RA) 4
1.5. Recursos (Equipos, materiales, programas y otros) 4
2. Procedimiento o Metodología 4
3. Conclusiones (especifique por lo menos 3) 13
4. Cuestionario 13
5. Referencias Bibliográficas 14

Introducción

En el desarrollo de aplicaciones Android modernas, la persistencia de datos es un pilar fundamental para ofrecer una experiencia de usuario fluida, permitiendo que la información se mantenga disponible incluso sin conexión a internet o tras cerrar la aplicación. Room Database se presenta como una capa de abstracción sobre SQLite que facilita el acceso fluido a la base de datos, aprovechando toda la potencia de SQLite mientras proporciona una verificación de consultas en tiempo de compilación.

Este laboratorio práctico se centra en la arquitectura de persistencia recomendada por Google. Los estudiantes aprenderán a estructurar una base de datos local utilizando los tres componentes clave de Room:

1. Entity: Que representa las tablas de la base de datos.
2. DAO (Data Access Object): Que define los métodos para acceder a los datos.
3. Database: Que sirve como el punto de acceso principal para la conexión.

Además, se integrará el uso de Corrutinas de Kotlin y el componente ViewModel para asegurar que las operaciones de entrada/salida (I/O) se realicen de forma asíncrona, manteniendo la interfaz de usuario reactiva y libre de bloqueos..

Guía de Laboratorio Nº 06 “Implementación de Room Database en Kotlin”

1. Información sobre el evento práctico

1.1. Título del evento práctico

Implementación de Room Database en Kotlin.

1.2. Objetivos

- Implementar Room como medio de almacenamiento interno de información.
- Integrar Workers con Corrutinas mediante el uso de Dispatchers.IO para optimizar el uso de hilos.

1.3. Tiempo de duración (horas)

2 Horas

1.4. Resultados de Aprendizaje (RA)

- El estudiante gestiona tareas de larga duración fuera del hilo principal, garantizando la persistencia de los procesos incluso si la aplicación se cierra.
- Aplica el principio de separación de responsabilidades delegando la lógica de trabajo pesado a Workers especializados.

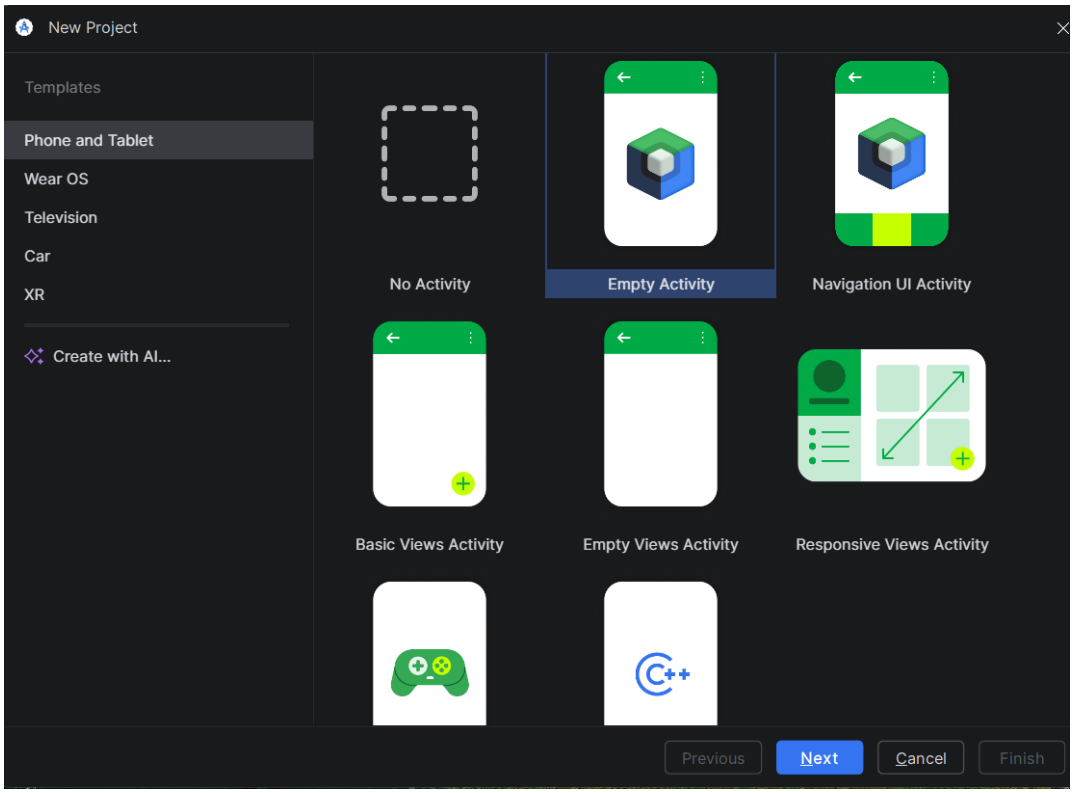
1.5. Recursos (Equipos, materiales, programas y otros)

- Computadora con acceso a internet.
- Android Studio (Versión más reciente).
- Dispositivo físico Android o creación de un Android Virtual Device (AVD).

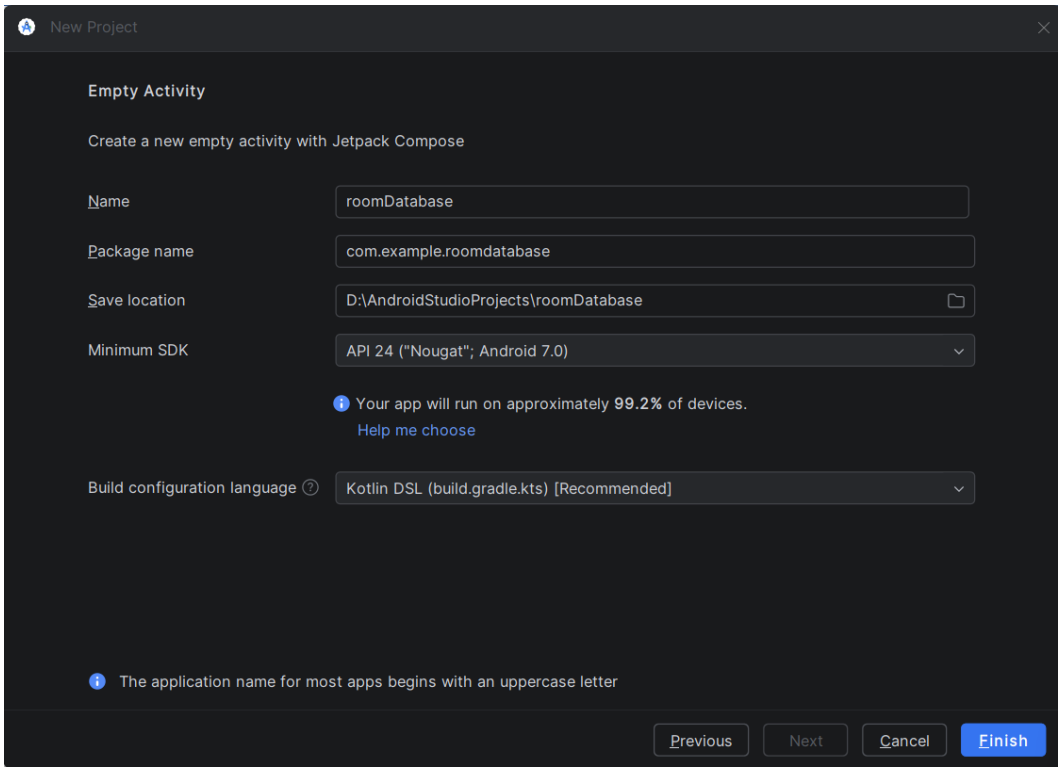
2. Procedimiento o Metodología

Inicio de la aplicación

Creamos un nuevo proyecto en Adroid Studio con una empty activity:



Colocamos de nombre roomDatabase:



Clases a trabajar y disposición en la aplicación:

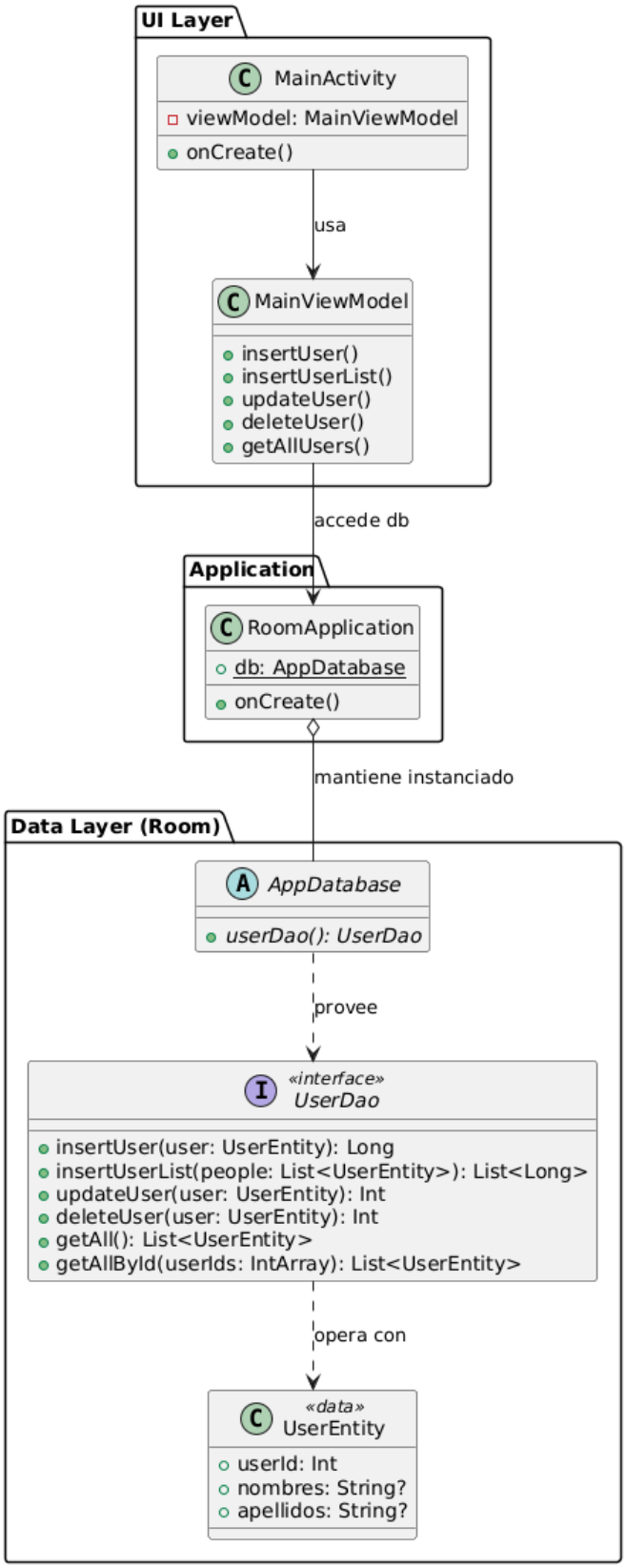
Se propone trabajar la siguiente distribución de clases para cubrir la sencilla lógica de un mantenimiento de tablas a traves de las bibliotecas Room:

```
@startuml

package "UI Layer" {
  class MainActivity {
    - viewModel: MainViewModel
    + onCreate()
  }
  class MainViewModel {
    + insertUser()
    + insertUserList()
    + updateUser()
    + deleteUser()
    + getAllUsers()
  }
}
package "Application" {
  class RoomApplication {
    {static} + db: AppDatabase
    + onCreate()
  }
}
package "Data Layer (Room)" {
  abstract class AppDatabase {
    + {abstract} userDao(): UserDao
  }

  interface UserDao <<interface>> {
     + insertUser(user: UserEntity): Long
     + insertUserList(people: List<UserEntity>): List<Long>
     + updateUser(user: UserEntity): Int
     + deleteUser(user: UserEntity): Int
     + getAll(): List<UserEntity>
     + getAllById(userIds: IntArray): List<UserEntity>
  }

  class UserEntity <<data>> {
    + userId: Int
    + nombres: String?
    + apellidos: String?
  }
}
' Relationships
MainActivity --> MainViewModel : usa
MainViewModel --> RoomApplication : accede db
RoomApplication o-- AppDatabase : mantiene instanciado
AppDatabase ..> UserDao : provee
UserDao ..> UserEntity : opera con
@enduml
```


UI Layer
MainActivity
viewModel: MainViewModel
onCreate()
usa
MainViewModel
insertUser()
insertUserList()
updateUser()
deleteUser()
getAllUsers()
accede db
Application
RoomApplication
db: AppDatabase
onCreate()
mantiene instanciado
Data Layer (Room)
AppDatabase
userDao(): UserDao
provee
«interface»
UserDao
insertUser(user: UserEntity): Long
insertUserList(people: List<UserEntity>): List<Long>
updateUser(user: UserEntity): Int
deleteUser(user: UserEntity): Int
getAll(): List<UserEntity>
getAllById(userIds: IntArray): List<UserEntity>
opera con
«data»
UserEntity
userId: Int
nombres: String?
apellidos: String?

Iniciamos con la configuración de los scripts de la aplicación, de modo tal que podamos sincronizar el uso de las bibliotecas de Android:

Para esto modificamos el fichero “libs.versions.toml” agregando las versiones del compilador de la siguiente forma:

```
[versions]…

kotlin = "2.3.21"
composeBom = "2026.04.01"
kotlinskp = "2.3.4"…


[plugins]
android-application = { id = "com.android.application", version.ref = "agp" }
kotlin-compose = { id = "org.jetbrains.kotlin.plugin.compose", version.ref = "kotlin" }
kotlinksp = {id= "com.google.devtools.ksp", version.ref = "kotlinskp"}
```

Luego modificamos el fichero “build.gradle.kts” del proyecto de la siguiente forma:

```
plugins {
  …
  alias(libs.plugins.kotlinksp) apply false
}
```

Finalmente modificamos el fichero “build.gradle.kts” de la aplicación:

```
plugins {
  …
  alias(libs.plugins.kotlinksp)
}…

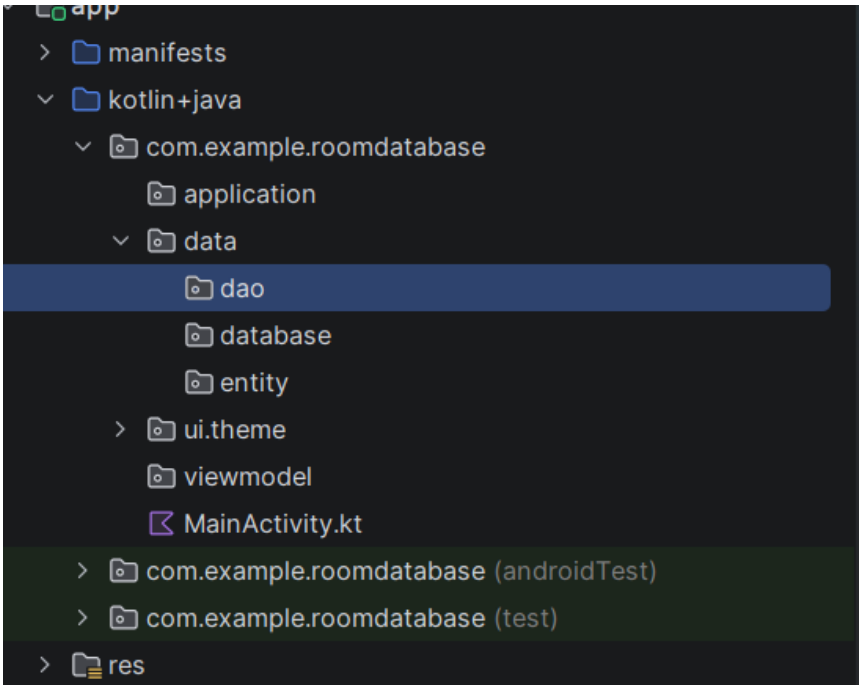

dependencies {
  val room_version = "2.8.4"

    implementation("androidx.room:room-runtime:$room_version")

    ksp("androidx.room:room-compiler:$room_version")
    annotationProcessor("androidx.room:room-compiler:$room_version")
    implementation("androidx.room:room-ktx:$room_version")…

}
```

Ahora vamos a generar los paquetes necesarios para poder comprender el funcionamiento de las clases del proyecto, los cuales son: application, data{dao,database,entity}, ui y viewmodel.



Ahora procedemos a crear la data class UserEntity que será la representación de la tabla “users”:

```
package com.example.roomdatabase.data.entity

import androidx.room.ColumnInfo
import androidx.room.Entity
import androidx.room.PrimaryKey

@Entity(tableName = "users")
data class UserEntity(
  @PrimaryKey(autoGenerate = true) val userId: Int = 0,
  @ColumnInfo(name = "first_name") val nombres: String?,
  @ColumnInfo(name = "last_name") val apellidos: String?
)
```

Luego procedemos a definir la interface que permitirá trabajar con la entidad, esto a través de la interface UserDao en el paquete dao:

```
package com.example.roomdatabase.data.dao

import androidx.room.Dao
import androidx.room.Delete
import androidx.room.Insert
import androidx.room.OnConflictStrategy
```

```
import androidx.room.Query
import androidx.room.Update
import com.example.roomdatabase.data.entity.UserEntity

@Dao
interface UserDao {
   @Insert(onConflict = OnConflictStrategy.ABORT)
   suspend fun insertUser(user: UserEntity):Long
   @Insert(onConflict = OnConflictStrategy.ABORT)
   suspend fun insertUserList(people: List<UserEntity>): List<Long>
   @Update
   suspend fun updateUser(user: UserEntity): Int
   @Delete
   suspend fun deleteUser(user: UserEntity): Int
   @Query("SELECT * FROM users")
   suspend fun getAll(): List<UserEntity>
   @Query("SELECT * FROM users WHERE userId IN (:userIds)")
   suspend fun getAllById(userIds: IntArray): List<UserEntity>?
}
```

Para orquestar con la base de datos, procederemos a definirla a través de la clase abstracta, la cual llamaremos AppDatabase, la cual heredará los métodos de RoomDatabase:

```
package com.example.roomdatabase.data.database

import androidx.room.Database
import androidx.room.RoomDatabase
import com.example.roomdatabase.data.dao.UserDao
import com.example.roomdatabase.data.entity.UserEntity

@Database(entities = [UserEntity::class], version = 1)
abstract class AppDatabase : RoomDatabase() {
  abstract fun userDao(): UserDao
}
```

Para poder tener una instancia única de la base de datos, vamos a crear una clase que herede de Application con companion object, que tendrá dicho propósito:

```
package com.example.roomdatabase.application

import android.app.Application
import androidx.room.Room
import com.example.roomdatabase.data.database.AppDatabase

class RoomApplication: Application (){
  companion object {
    lateinit var db: AppDatabase
  }

  override fun onCreate() {
    super.onCreate()
    db = Room.databaseBuilder(
```

```
        applicationContext,
        AppDatabase::class.java,
        "database-users").build()
  }
}
```

Para completar la llamada a esta última clase, vamos a modificar el fichero AndroidManifest.xml, indicando el nombre de la aplicación con la clase:

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
  xmlns:tools="http://schemas.android.com/tools">

    <application
      android:name=".application.RoomApplication"
      android:allowBackup="true"
      android:dataExtractionRules="@xml/data_extraction_rules"
      android:fullBackupContent="@xml/backup_rules"
      android:icon="@mipmap/ic_launcher"
      android:label="@string/app_name"
      android:roundIcon="@mipmap/ic_launcher_round"
      android:supportsRtl="true"
      android:theme="@style/Theme.RoomDatabase">
      <activity
        android:name=".MainActivity"
        android:exported="true"
        android:label="@string/app_name"
        android:theme="@style/Theme.RoomDatabase">
        <intent-filter>
           <action android:name="android.intent.action.MAIN" />

           <category android:name="android.intent.category.LAUNCHER" />
        </intent-filter>
      </activity>
    </application>

</manifest>
```

Ahora se debe crear el ViewModel, que nos servirá para la ejecución de las instrucciones que gestionan la base de datos:

```
package com.example.roomdatabase.viewmodel

import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import com.example.roomdatabase.application.RoomApplication
import com.example.roomdatabase.data.entity.UserEntity
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.launch

class MainViewModel: ViewModel() {
  fun insertUser() {
    val userEntity = UserEntity(nombres = "Juan", apellidos = "Perez")
```

```
    viewModelScope.launch(Dispatchers.IO){
       RoomApplication.db.userDao().insertUser(userEntity)
    }
  }
  fun insertUserList() {
    val users = listOf(
       UserEntity(nombres = "Juan", apellidos = "Perez"),
       UserEntity(nombres = "Maria", apellidos = "Lopez"),
       UserEntity(nombres = "Pedro", apellidos = "Gonzalez"),
       UserEntity(nombres = "Ana", apellidos = "Martinez"),
       UserEntity(nombres = "Luis", apellidos = "Garcia")
    )
    viewModelScope.launch(Dispatchers.IO){
       RoomApplication.db.userDao().insertUserList(users)
    }
  }
}
```

Ahora para probar nuestro ViewModel, vamos a agregarlo en el fichero MainActivity.kt:

```
class MainActivity : ComponentActivity() {

    private val viewModel:MainViewModel by viewModels()
    override fun onCreate(savedInstanceState: Bundle?) {
      super.onCreate(savedInstanceState)

      viewModel.insertUser()
      viewModel.insertUserList()

      enableEdgeToEdge()
   …
```

Para completar la solución, modificamos el fichero MainViewModel, otorgando la funcionalidad de los métodos que faltan:

```
fun updateUser(){
     val userEntity = UserEntity(userId = 1, nombres = "Jorge", apellidos = "Perez")
     viewModelScope.launch(Dispatchers.IO){
        RoomApplication.db.userDao().updateUser(userEntity)
        println("DEBUG: Usuario actualizado correctamente")
     }
  }
  fun deleteUser() {
     val userEntity = UserEntity(userId = 4,"","")
     viewModelScope.launch(Dispatchers.IO) {
        RoomApplication.db.userDao().deleteUser(userEntity)
     }
  }
  fun getAllUsers(){
     viewModelScope.launch(Dispatchers.IO){
        val users = RoomApplication.db.userDao().getAll()
        users.forEach { user ->
```

```
            println("DEBUG: ${user.userId} - ${user.nombres} - ${user.apellidos}")
        }
    }
}
```

Finalmente, agregamos las instrucciones al MainActivity:

```
private val viewModel:MainViewModel by viewModels()
override fun onCreate(savedInstanceState: Bundle?) {
  super.onCreate(savedInstanceState)

    viewModel.updateUser()
    viewModel.deleteUser()
    viewModel.getAllUsers()
```

3. Conclusiones (especifique por lo menos 3)

4. Cuestionario

1. ¿Cuál es la función principal de la anotación @Dao y qué ventaja ofrece sobre el uso directo de SQLiteOpenHelper?
2. En la clase UserEntity, se utilizó @PrimaryKey(autoGenerate = true). Explique qué sucede si se intenta insertar un objeto con el userId inicializado en 0.
3. ¿Por qué es necesario ejecutar las funciones del DAO dentro de un bloque viewModelScope.launch(Dispatchers.IO)? ¿Qué sucedería si se ejecutan en el hilo principal?
4. Explique el propósito de crear una clase que hereda de Application (como RoomApplication) para inicializar la base de datos.
5. ¿Qué función cumple el motor KSP (Kotlin Symbol Processing) configurado en los archivos build.gradle.kts en relación con Room?

Enunciado

Una vez implementada la estructura básica de UserEntity y su respectivo mantenimiento, se requiere escalar la aplicación para que sea funcional para un usuario final, añadiendo validaciones y una visualización de datos más robusta.

1. **Modificación de la Entidad:**
   - Agregue un nuevo campo a la clase UserEntity llamado email de tipo String (debe permitir valores nulos).
   - Actualice la versión de la base de datos en AppDatabase a la versión 2 y configure una política de migración destructiva básica (`.fallbackToDestructiveMigration()`) en el Room.databaseBuilder.
2. **Implementación de Búsqueda:**
   - Añada un nuevo método en UserDao que permita buscar usuarios por su apellido (last_name) utilizando la cláusula LIKE de SQL.
   - Implemente la función correspondiente en el MainViewModel.
3. **Integración de UI (Interfaz de Usuario):**
   - En el MainActivity, diseñe una interfaz sencilla (usando Jetpack Compose) que contenga:
     - Dos campos de texto (Nombre y Apellido).
     - Un botón para ”Guardar Usuario”.
     - Un componente de lista (LazyColumn o RecyclerView) que muestre los usuarios guardados actualmente en la base de datos.
4. **Lógica de Negocio:**
   - Asegúrese de que la lista de usuarios se actualice automáticamente en la pantalla cada vez que se inserte o elimine un registro.

5. Referencias Bibliográficas

- Android Developers (2026). Ciclo de vida de la actividad. Recuperado de la documentación oficial de Android.
- Android Developers (2026). Guía de WorkManager: Programación de tareas asíncronas. Recuperado de la documentación oficial de Android Jetpack.
- Kotlin Foundation (2025). Coroutines Guide: Dispatchers and Context.
- Android Developers (2026). Common Architecture Patterns: MVVM with Repository and Jetpack Work.